

Web Applications

TypeScript

Suhel Hammoud

TypeScript for JavaScript Programmers

TypeScript extends JavaScript by adding a powerful type system. It catches errors early, making your code more reliable.

- **JavaScript Primitives:** `string`, `number`, etc., but no type checking.
- **TypeScript:** Adds type checking to JavaScript.
- **Benefit:** Highlights unexpected behavior, reducing bugs.

Types by Inference

TypeScript infers types automatically. For example:

```
let helloWorld = "Hello World"; // TypeScript infers `helloWorld` as `string`.
```

- No need to explicitly declare types in many cases.
- Works seamlessly with tools like Visual Studio Code.

Defining Types

When TypeScript can't infer types, you can define them explicitly.

Example: Inferred Object

```
const user = {  
  name: "Hayes",  
  id: 0,  
};
```

Example: Explicit Interface

```
interface User {  
  name: string;  
  id: number;  
}  
  
const user: User = {  
  name: "Hayes",  
  id: 0,  
};
```

TypeScript Warnings

TypeScript warns if objects don't match the interface:

```
interface User {  
  name: string;  
  id: number;  
}  
  
const user: User = {  
  username: "Hayes", // Error: 'username' does not exist in type 'User'.  
  id: 0,  
};
```

Classes and Interfaces

TypeScript supports classes and interfaces together:

```
interface User {  
  name: string;  
  id: number;  
}  
  
class UserAccount {  
  constructor(public name: string, public id: number) {}  
}  
  
const user: User = new UserAccount("Murphy", 1);
```

Function Annotations

Interfaces can annotate function parameters and return types:

```
function deleteUser(user: User) {  
    // ...  
}  
  
function getAdminUser(): User {  
    //...  
}
```

Primitive Types

TypeScript extends JavaScript's primitives with `any`, `unknown`, `never`, and `void`.

- Prefer `interface` for defining types.
- Use `type` for advanced features.

Composing Types

Combine types using **unions** and **generics**.

Unions

Declare a type as one of many:

```
type MyBool = true | false;  
type WindowStates = "open" | "closed" | "minimized";
```

Example: Union in Functions

```
function getLength(obj: string | string[]) {  
    return obj.length;  
}
```

Using typeof

Check types at runtime:

```
function wrapInArray(obj: string | string[]) {  
  if (typeof obj === "string") {  
    return [obj];  
  }  
  return obj;  
}
```

Generics

Generics make types reusable:

```
type StringArray = Array<string>;  
type NumberArray = Array<number>;
```

Example: Custom Generics

```
interface Backpack<Type> {  
  add: (obj: Type) => void;  
  get: () => Type;  
}  
  
declare const backpack: Backpack<string>;  
const object = backpack.get();  
backpack.add(23); // Error: 'number' is not assignable to 'string'.
```

■ Structural Type System

TypeScript checks the **shape** of objects, not their explicit type.

▨ Example: Structural Typing

```
interface Point {  
  x: number;  
  y: number;  
}  
  
function logPoint(p: Point) {  
  console.log(`${p.x}, ${p.y}`);  
}  
  
const point = { x: 12, y: 26 };  
// Works even if `point` isn't explicitly a `Point`.  
logPoint(point); //12 26
```

Example: Shape Matching

Only required properties need to match:

```
function logPoint(p: Point) {  
  console.log(`${p.x}, ${p.y}`);  
}  
  
const point3 = { x: 12, y: 26, z: 89 };  
logPoint(point3); // Works: `x` and `y` match.  
  
const color = { hex: "#187ABF" };  
logPoint(color); // Error: Missing `x` and `y`.
```

Example: Classes and Structural Typing

Classes also conform to shapes:

```
class VirtualPoint {  
  constructor( x: number, y: number) {  
    this.x = x;  
    this.y = y;  
  }  
}  
  
const newVPoint = new VirtualPoint(13, 56);  
logPoint(newVPoint); // Works: `x` and `y` match.
```

Conclusion

TypeScript enhances JavaScript with:

- A robust type system.
- Better tooling and error detection.
- Advanced features like generics and unions.

References:

- <https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html>